Elijah Salgado
CSC432
Dr. Benjamin Knisely
Midterm report

Security: Network Traffic Anomaly Detection with ML

Network traffic anomaly detection is a technique used to monitor, analyze, and identify unusual patterns or activities within a computer network. These anomalies are deviations from normal behavior which could indicate potential security threats, such as cyberattacks or unauthorized access. Identifying these anomalies are important in order to help prevent such security threats from occurring. My goal was to create a binary classification of BENIGN vs ATTACK traffic using Machine Learning (ML). In this project, I used the Canadian Institute for Cybersecurity Intrusion Detection System 2017 (CICIDS2017) dataset which contains real-world benign and the common attacks (PCAPs) that occurred up to 2017. I then trained and compared two ML models: Random Forest and Support Vector Machine (SVM).

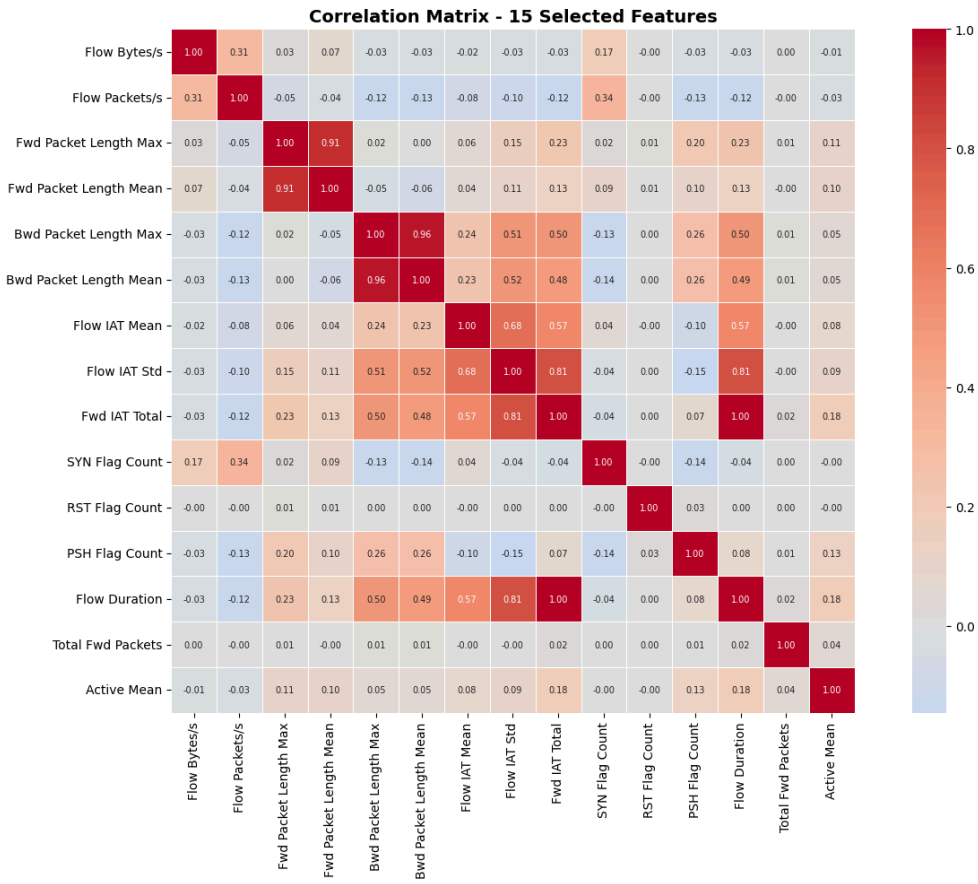
CICIDS2017 was generated in a controlled lab environment simulating a real enterprise network over 5 days. It includes labeled benign traffic, a variety of cyber attacks, ~80 features extractor from raw pcap files using CICFlowMeter, and modern protocols (HTTP, HTTPS, FTP, SSH, email). I used three cvs files: *Monday-WorkingHours.pcap_ISCX.csv, Friday-WorkingHours-Afternoon-DDos.pcap_ISCX.csv, and Wednesday-WorkingHours.pcap_ISCX.csv*. I chose these files because the files from Monday only contain the benign baseline, Wednesday contains DoS, and Friday contains DDos attacks. There was a class imbalance problem; Heartbleed only had 11 samples which is statistically unlearnable. So, I had to drop those samples. Once dropped, I had to clean the dataset because real-world data always has problems. There were issues with negative values, Nan Values (missing data) and duplicate rows. After cleaning the data, I was left with 1,333,264 rows of data. This data was separated using binary label encoding which had us with samples being 75.9% benign and 24.1% attacks.

I then selected 15 features to categorize the attacks in order for the ML models to learn from their characteristics: Flow Bytes/s, Flow Packets/s, Fwd Packet Length Max, Fwd Packet Length Mean, Bwd Packet Length Max, Bwd Packet Length Mean, Flow IAT Mean, Flow IAT Std, SYN Flag Count, RST Flag Count, PSH Flag Count, Flow Duration, Total Fwd Packets, Active Mean, and Init_Win_bytes_forward. These features were chosen based on domain knowledge, known importance in CICIDS2017 literature, and their ability to cover different angles of traffic behavior. They fall into five categories: Volume, Packet Size, Timing, TCP Flags, and Connection.

Flow Bytes/s and Flow Packets/s represent the Volume category and capture how much data is being transferred per second. DoS floods produce extreme packet rates, making these features strong indicators of attack behavior. The Packet Size category includes Fwd Packet Length Max, Fwd Packet Length Mean, Bwd Packet Length Max, and Bwd Packet Length Mean. These measure the size of packets traveling in both directions. Attack packets are often abnormally small or uniform because automated tools do not carry real content the way legitimate traffic does. The Timing category consists of Flow IAT Mean and Flow IAT Std, which measure the inter-arrival time between packets and its variability. Flood attacks have near-zero inter-arrival time while slow attacks like Slowloris deliberately space packets far

apart, meaning this category captures both attack styles. The TCP Flags category includes SYN Flag Count, RST Flag Count, and PSH Flag Count. These track how TCP control flags are used within a flow, since attacks frequently abuse the handshake process or generate abnormal flag patterns. Finally, the Connection category covers Flow Duration, Total Fwd Packets, Active Mean, and Init_Win_bytes_forward. Flow Duration captures how long a session lasts, which differs significantly between normal traffic and slow attacks that hold connections open. Total Fwd Packets reflect the sheer volume of packets sent in one direction during a flood. Active Mean measures how long the flow was actively transmitting. Init_Win_bytes_forward captures the initial TCP window size advertised by the sender. Legitimate clients typically advertise standard window sizes, while attackers often use tiny or zero window sizes because they are not genuinely trying to receive data.

During initial feature selection, Fwd IAT Total was included but was later dropped after the correlation analysis revealed a correlation of 1.00 with Flow Duration, meaning the two features were effectively measuring the same thing. Keeping both would have given the model redundant signal without adding any new information. Init_Win_bytes_forward was added as a replacement because it contributes unique information about TCP behavior that none of the other 14 features capture. See Figure 1 for the full correlation matrix across all 15 selected features.

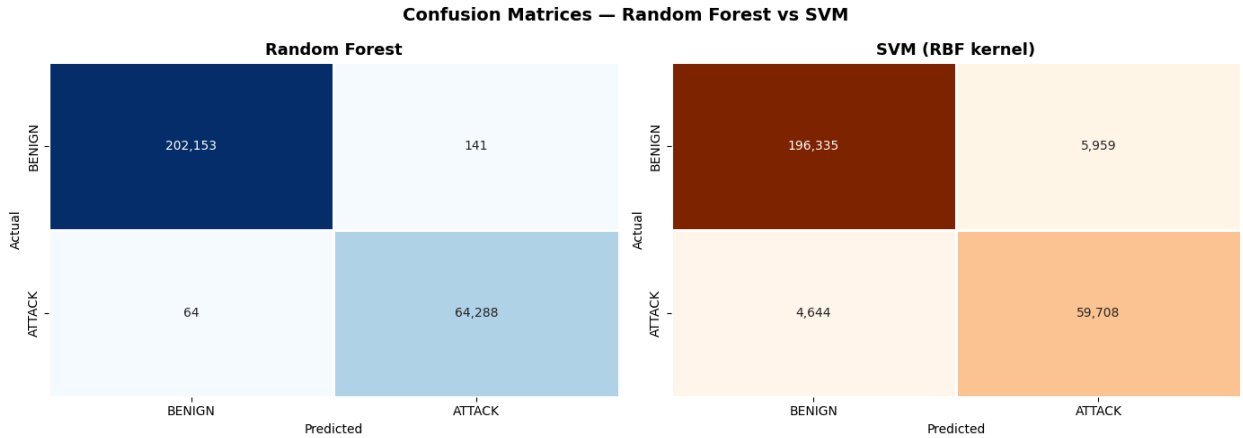


[Figure 1: Correlation Heatmap of the 15 Features]

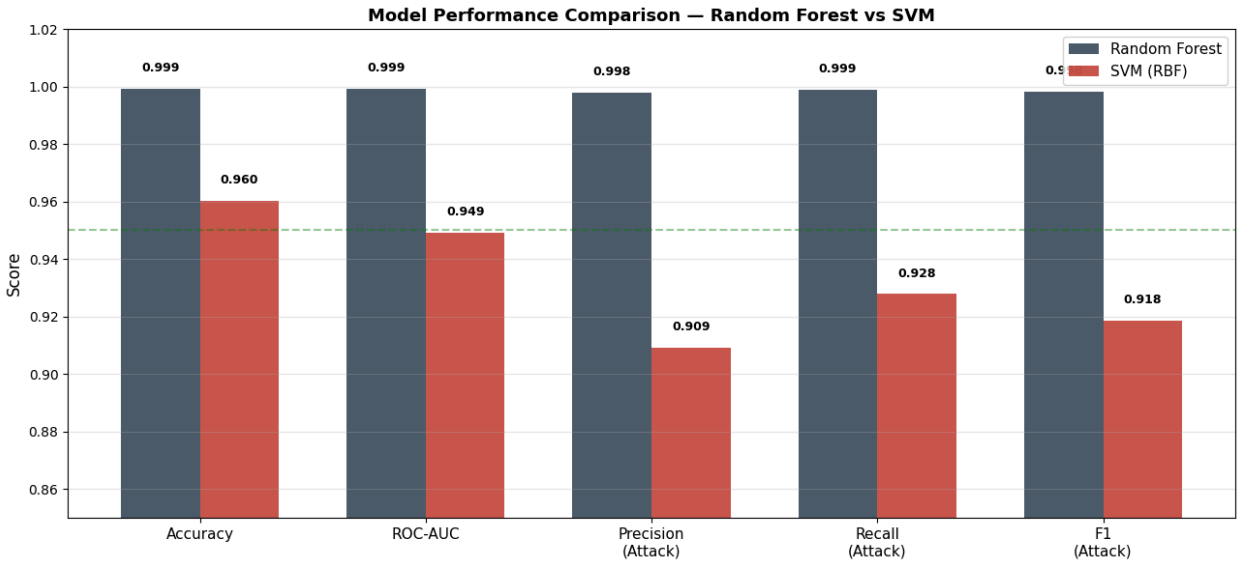
To prepare the data for modeling, the 1,333,264 clean rows were split into a training set and a test set using an 80/20 ratio, resulting in 1,066,582 training rows and 266,646 test rows. Stratified splitting was used to ensure that the 24.1% attack rate was preserved in both the training and test sets, preventing the model from being evaluated on a class distribution that does not reflect the data it was trained on. All features were then scaled using StandardScaler, which centers each feature to a mean of zero and scales it to unit variance. It is critical that the scaler is fit only on the training data and then applied to the test data separately. Fitting the scaler on the entire dataset before splitting would allow test data to influence the scaling parameters, which is a form of data leakage that causes artificially inflated accuracy scores.

Random Forest was trained on the full training set using 100 decision trees. Each tree is built on a random subset of the data and features, and the final prediction is determined by a majority vote across all trees. The class_weight parameter was set to balanced so that the model accounts for the unequal distribution of benign and attack samples. Support Vector Machine was trained using a Radial Basis Function (RBF) kernel, which allows the model to find a non-linear decision boundary separating the two classes in high-dimensional space. Because SVM is memory-intensive and does not scale efficiently to over one million rows, it was trained on a stratified sample of 100,000 rows rather than the full training set. This sampling was a practical necessity given available compute resources, not a methodological choice, and it is important to keep in mind when comparing the two models.

Random Forest achieved an accuracy of 99.92% and a ROC-AUC of 99.92%. Its precision on the attack class was 99.78%, recall was 99.90%, and F1 score was 99.84%. The false negative rate was 0.10%, meaning only 64 out of 64,352 actual attacks in the test set were missed. SVM achieved an accuracy of 96.02% and a ROC-AUC of 94.92%. Its precision on the attack class was 90.94%, recall was 92.78%, and F1 score was 91.85%. The false negative rate was 7.22%, meaning 4,644 attacks were not detected. See Figure 2 for the confusion matrices for both models and Figure 3 for the side-by-side metric comparison.



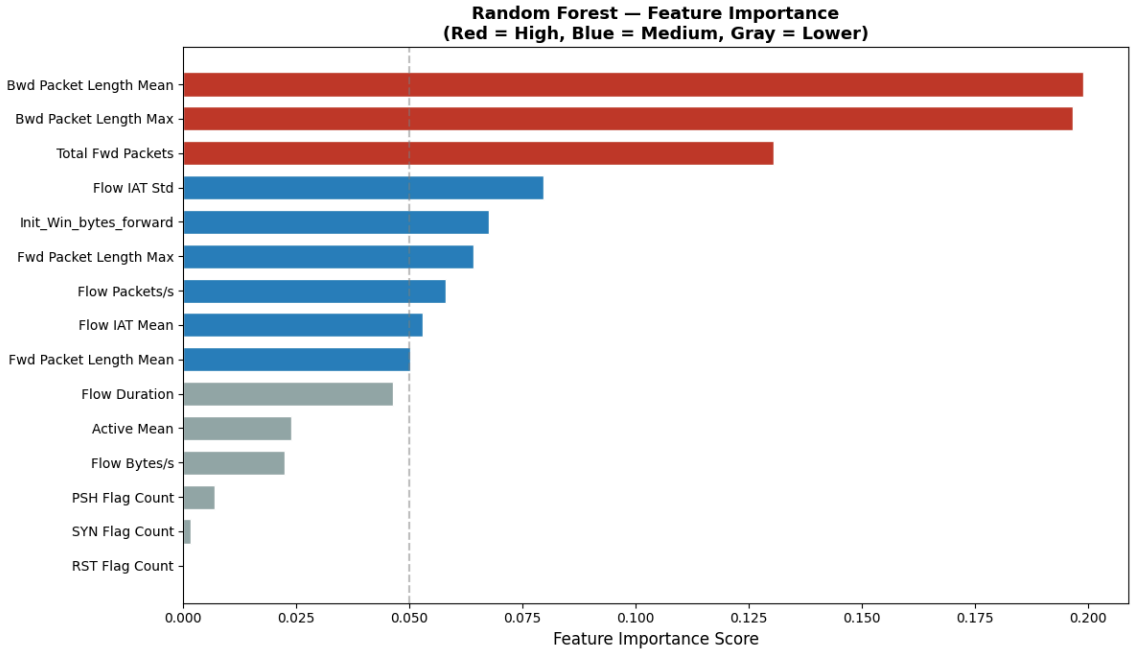
[Figure 2: Confusion Matrices (Random Forest vs SVM)]



[Figure 3: Model Comparison]

In a real intrusion detection system, the false negative rate is the most operationally significant metric because a missed attack is far more dangerous than a false alarm. Random Forest's false negative rate of 0.10% versus SVM's 7.22% represents a substantial difference in practice. At SVM's miss rate, a network processing this volume of traffic would allow thousands of genuine attacks to pass undetected every day.

The Random Forest feature importance scores revealed that the two most influential features were Bwd Packet Length Mean and Bwd Packet Length Max, each contributing approximately 0.20 out of a total importance of 1.0. This was the most significant finding of the analysis. Rather than raw outgoing volume, the strongest signal of an attack was found in how the server responded. When a DoS attack overwhelms a server, its ability to send normal replies degrades. Backward packet sizes shrink or disappear entirely, and the model learned to detect that degradation more reliably than it detected the volume of incoming traffic. Total Fwd Packets ranked third at 0.13, capturing the massive unidirectional packet counts characteristic of flood attacks. Flow IAT Std and Init_Win_bytes_forward followed as the fourth and fifth most important features respectively. See Figure 4 for the full feature importance ranking.



[Figure 4: Feature Importance]

SYN Flag Count ranked fourteenth with an importance score near zero, which appears counterintuitive given that SYN floods are one of the most well-known attack types. However, this result is explained by the composition of the dataset. The attacks present in the three files used, primarily DoS Hulk, DDoS, and Slowloris, do not rely on SYN flooding as their primary mechanism. DoS Hulk sends legitimate-looking HTTP GET requests, and Slowloris holds connections open using partial HTTP headers. Neither attack type generates an abnormal SYN flag count, so the feature provided little discriminating power for this particular dataset. This does not mean SYN Flag Count is unimportant in general; it would likely rank much higher on a dataset containing port scan or SYN flood traffic.

Random Forest clearly outperformed SVM across every metric, but several honest limitations must be acknowledged. First, the comparison between the two models is not entirely fair. SVM was trained on only 100,000 rows, approximately 10% of the available training data, while Random Forest used the full 1,066,582 rows. A fair comparison would require training SVM on the same volume of data, which would demand significantly more compute time and memory. Second, the CICIDS2017 dataset was generated in 2017, and modern attack signatures have evolved considerably since then. Techniques such as encrypted malicious traffic, adversarial evasion, and low-and-slow attacks designed to mimic normal behavior may not be captured by features derived from this dataset. Third, the binary classification approach used here collapses all attack types into a single label, which hides information about what kind of attack is occurring. A deployed system would benefit from knowing whether a flow is a DoS attack, a DDoS attack, or an infiltration attempt. Finally, high accuracy on a controlled benchmark dataset does not guarantee equivalent performance in a live network environment, where traffic distributions, protocol mixes, and attacker behavior differ from the lab conditions under which CICIDS2017 was collected.

Both models exceeded the 95% accuracy goal set for this project, with Random Forest achieving near-perfect detection at 99.92% accuracy and SVM reaching 96.02%. The most important finding was that backward packet length features, specifically how the server responds under attack, were more discriminating than raw volume metrics like Flow Bytes/s. This insight demonstrates that anomaly detection is not simply about measuring how much traffic arrives, but about understanding how the entire flow behaves in both directions. Future work on this system could pursue multiclass classification to distinguish between specific attack types, development of a real-time detection pipeline that scores live traffic rather than pre-labeled flows, and evaluation on newer datasets that reflect the current threat landscape beyond 2017.